

Note on how to read this:

This is a completely delineated list of all content covered in the lectures and coding labs. It may look daunting, but if you understand the lecture materials, can explain how, e.g. gradient descent for multivariate linear regression and K-means clustering and PCA works step-by-step, you are probably quite close to getting a good grade. Note that all lecture content can be on the exam, as can (the logic/pseudocode of) implementations, as can be scikit-learn pipelines (to the extent covered in the lectures on Day 6 and the Project).

Day 1

- Can describe the difference between supervised and unsupervised machine learning
- Can describe the difference between classification and regression
- Can explain GWAS as (many) linear regressions on variants (“a bunch of linear regressions in a trenchcoat”)
- Can write down the formula for univariate linear regression:

$$\theta_0 + \theta_1 \cdot x^{(i)}$$

- Can describe what samples and features are in data for Machine Learning
- Can describe how we learn parameters from training data (using a cost function and gradient descent)
- Can explain what a cost function measures
- Can write down the cost function for linear regression (mean-squared error):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y_{\text{pred}}^{(i)} - y_{\text{real}}^{(i)})^2$$

where

$$y_{\text{pred}}^{(i)} = \theta_0 + \theta_1 \cdot x^{(i)}$$

- Can describe how trying different values for linear regression parameters and seeing what the MSE is shows you what the optimal parameters are
- Can explain that just trying all possible parameter combinations is impossible
- Can explain that gradient descent is the solution: take tiny steps towards the minimum of an unknown cost function

- Can describe how to calculate a gradient:

$$\Delta y / \Delta x$$

and take a small step towards the minimum:

$$\text{current_param} - \alpha \cdot \text{gradient}$$

- Can describe that iteratively taking small steps towards the minimum of the cost function will eventually yield good parameters
- Can explain why we need to use partial derivatives rather than a single gradient
- Can explain that taking the partial derivative of a function just means treating all parameters except one as constants
- Can calculate partial derivatives of univariate/multivariate linear regression cost function
- Can use the chain rule to calculate partial derivatives. Example (for MSE):

$$\frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 \cdot x^{(i)} - y_{\text{real}}^{(i)})^2$$

Partial derivative w.r.t. θ_0 :

$$\frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 \cdot x^{(i)} - y_{\text{real}}^{(i)})$$

- Can describe that α or η is a hyperparameter that governs step size in gradient descent
- Can describe that hyperparameters are not optimized by the ML algorithm itself

<coding_lab_1 (Day 1 short 1)>

- Can implement the MSE function for univariate linear regression
- Explain what the 3D surface plot of the univariate linear regression cost function means
- Can implement the gradient descent function for univariate linear regression
- Explain what a contour plot of a cost function shows
- Can explain that since θ_1 's partial derivative is multiplied with the feature value while θ_0 's is not, normalizing features allows equal step sizes

- Can explain how to extend univariate to multivariate regression:

$$\theta_0 + \theta_1 \cdot x_1^{(i)} + \theta_2 \cdot x_2^{(i)}$$

or more generally:

$$\hat{y}^{(i)} = \theta_0 + \sum_{j=1}^n \theta_j \cdot x_j^{(i)}$$

- Can write down partial derivatives for multivariate regression:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y_{\text{real}}^{(i)})^2$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y_{\text{real}}^{(i)})$$

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y_{\text{real}}^{(i)}) x_j^{(i)} \quad \text{for } j = 1, \dots, n$$

- Can explain polynomial features (raising inputs to powers)
- Can explain why standardizing features helps gradient descent converge
- Can explain overfitting vs underfitting, bias vs variance
- Can explain and draw examples of high-bias and high-variance fits
- Can explain train/validation/test splits, k-fold cross-validation, and learning curves
- Can explain why k-fold CV is preferred over a single split
- Can explain the learning curve differences for high bias vs high variance

<coding_lab_2 (Day 1 short 2)>

- Can add polynomial features
- Can explain scaling differences:

Z-score:

$$x' = \frac{x - \mu}{\sigma}$$

Range scaling:

$$x' = a + \frac{(x - x_{\min})}{(x_{\max} - x_{\min})} (b - a)$$

- Can implement multivariate linear regression and gradient descent for any number of parameters
-

- Can explain why we use linear algebra for ML
- Can perform matrix and vector operations
- Can determine when matrices can be multiplied (e.g., 5×3 with $3 \times 1 \rightarrow 5 \times 1$)
- Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & 4 \\ 2 & 0 & 1 \\ -3 & 5 & 2 \\ 4 & 1 & -2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$$

$$A\mathbf{x} = \begin{bmatrix} 2 \\ -6 \\ 1 \\ 5 \\ 8 \end{bmatrix}$$

- Can explain non-commutativity of matrix multiplication
 - Can cast linear regression as matrix-vector multiplication
-

<computer_lab_3 (Day 1 afternoon)>

- Can explain 9-fold CV on polynomial regression
 - Can do linear algebra by hand and verify with NumPy
 - Can implement regression hypothesis, MSE, and gradient descent using NumPy
-

Day 2

- Can explain why plain linear regression fails for classification
- Can define logistic regression:

$$h_{\theta}(x) = g(\theta^{\top}x), \quad g(z) = \frac{1}{1 + e^{-z}}$$

- Can interpret $h_\theta(x)$ as $p(y = 1 \mid x; \theta)$ and use $1 - h_\theta(x)$ for $y = 0$
- Can explain linear and nonlinear decision boundaries
- Can explain why MSE is poor for logistic regression and define the logistic loss:

$$\text{Cost}(x) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1, \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Dataset average:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y \log h_\theta(x^{(i)}) - (1 - y) \log(1 - h_\theta(x^{(i)}))]$$

- Gradient of $J(\theta)$ (!Note: unchanged save for sigmoid in hypothesis function!):

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- Gradient descent update:

$$\theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

<computer_lab_4 (Day 2 short 1)>

- Implement sigmoid, hypothesis, cost, and gradient descent for logistic regression
 - Plot sigmoid-based decision boundaries
-

- One-vs-rest multiclass logistic regression:

$$\hat{y} = \arg \max_i P(y = i \mid x; \theta^{(i)})$$

- Define TP, TN, FP, FN; sensitivity (TPR) and specificity (TNR)
- Explain threshold trade-offs, ROC curves, and AUC
- Explain precision-recall and when to prefer PRC (when you only value the positive/minority class)
- Discuss how performance metrics interact with biology and are used in reality: think cross-validation *across chromosomes*, stratifying by the distance genome bins are apart, etc.
- Write L2-regularized objective (excluding θ_0):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y \log h_\theta(x^{(i)}) - (1 - y) \log(1 - h_\theta(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- Explain bias–variance effects
 - Explain nested cross-validation (outer folds for generalization, inner for hyperparameter search)
 - Know that for your final classifier you use the hyperparameters that were picked most often, or the average of those picked for the outer folds.
-

<computer_lab_5 (Day 2 short 2)>

- Implement L2 regularization
 - Handle missing data (EDA, cleaning, KNN imputation)
 - Implement regularized logistic regression for multiple values
 - Implement ROC curve construction manually
-

- Explain self-supervised learning
 - Explain what neural networks and single neurons do
 - Explain forward pass via linear algebra
 - Explain one-hot encoding for multiclass NNs
 - Explain lack of inductive bias in MLPs and how CNNs/GNNs add it
-

<computer_lab_6 (Day 2 afternoon)>

- Implement multiclass logistic regression
- Predict test labels
- Implement nested cross-validation using scikit-learn
- Implement a basic feedforward neural network using linear algebra

Day 3

- Explain the idea of backpropagation: taking the cost from the output layer, through long chained partial derivatives, to get the gradients w.r.t the cost for all parameters.
- Explain that there's three core quantities to know per neuron: how the cost depends on its bias, its weights, and the activations it got from the previous layer.
- Write down the steps required (i.e. chained partial derivatives) to calculate the partial derivative of the cost w.r.t a certain parameter.

- Write down the analytical equations that belong to a sequence of partial derivatives. e.g.

$$\frac{\partial C}{\partial b_j^{(l)}} = \frac{\partial C}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} = \delta_j^{(l)} \cdot \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}}$$

- For instance, for mean-squared error, write out the full chain for the output layer bias:

$$C = \frac{1}{2}(y - a^{(L)})^2$$

$$\frac{\partial C}{\partial b^{(L)}} = \frac{\partial C}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial b^{(L)}}$$

Where:

$$\frac{\partial C}{\partial a^{(L)}} = -(y - a^{(L)})$$

$$\frac{\partial a^{(L)}}{\partial z^{(L)}} = \sigma'(z^{(L)})$$

$$\frac{\partial z^{(L)}}{\partial b^{(L)}} = 1$$

So:

$$\frac{\partial C}{\partial b^{(L)}} = -(y - a^{(L)}) \cdot \sigma'(z^{(L)})$$

(where σ is the activation function, e.g. sigmoid here)

<coding_lab_7 (Day 3 short 1)>

- Explain how backpropagation works

- Explain why convolutional neural networks make sense for images
- Explain how convolutions work (weighted sums that move over a sequence)
- Explain what stride, filter size, and the number of kernels/filters are
- Explain how you can use convolutions for a DNA sequence

<coding_lab_8 (Day 3 short 2)>

- Implement the cross-entropy cost function for a neural network, including regularisation
- Implement numerical gradient checking (brute-forcing the gradient calculation)
- Know the derivative of the sigmoid function: $\sigma something \cdot (1 - \sigma something)$

<computer_lab_9 (Day 3 afternoon)>

- Understand in-depth how backpropagation works
- Can implement backpropagation in linear algebra (note: not on exam!)

Day 4

- Explain that clustering is only possible with some (implicit) inductive bias on what is important
 - Explain that you always check whether your clustering makes sense in the light of external data/knowledge (e.g. known gene activities for subtypes, or known disease entities)
 - Explain how different distance metrics change what you consider close
 - Explain exactly how K-means clustering works (initialization, iterative cluster centroid and cluster assignment refinement, final cost). Can write detailed pseudocode for this.
 - Know that K-means clustering is initialization-dependent and so needs to be run multiple times
 - Know that each clustering has a global cost (the distortion, or average average square distance to the mean of each cluster for each point in that cluster)
-

<coding_lab_10 (Day 4 short 1)>

- Can implement K-means clustering and select the best clustering using the distortion.
-

- Understand exactly how hierarchical clustering works
 - Know what a linkage criterion is and what each of the (single, average, complete) linkage criterions implicitly assumes about cluster structure
 - Understand that since you calculate all-all distances this can be prohibitive for large datasets.
 - Can explain a little about graph clustering for single-cell datasets.
-

<coding_lab_11 (Day 4 short 2)>

- Implement complete and single linkage criteria
 - Cut hierarchical clustering at different levels.
 - Compare hierarchical and K-means clustering
-

Day 5

- Explain why high-dimensional data poses problems for supervised and unsupervised machine learning
 - Explain how filtering and wrapping work for feature selection
 - Explain the difference between non-linear and linear dimensionality reduction.
 - Explain the difference between extrinsic and intrinsic dimensionality
-

<coding_lab_12 (Day 5 short 1)>

- Implement classification with ever more noise features
 - Implement distance calculations in high-dimensional spaces to see that they become meaningless
-

- Explain exactly how PCA works, step by step:
 - Can explain what a covariance matrix captures
 - Can explain that we want to maximize the variance on each component
 - Can explain that it turns out this is an eigenvector-eigenvalue problem
 - Can explain that we can search for vectors that we need using the identity matrix and the fact that the determinant is 0 and solving for the eigenvalues
 - Can explain that we find vectors that satisfy the equations for the eigenvalues we found
 - Can explain that we limit them to unit length to get unique solutions.
 - Can explain that the percentage of variance captured on each component is the eigenvalue divided by the sum of all eigenvalues
 - Can explain how to project data onto the new axes through the data
 - Can explain how to finally perform dimension reduction by cutting of n dimensions such that you keep x% of total variance in the data
-

<coding_lab_13 (Day 5 short 2)>

- Can implement PCA yourself (using the covariance matrix and np.eig())
-

<coding_lab_14 (Day 5 afternoon)>

!Not on the exam this year! - Use K-means clustering for dimension reduction - Apply PCA to some actual datasets - Use PCA to filter out shared ancestry as a confounder in SNP analysis

Day 6

- Explain how scikit-learn fitting, predicting, and calculating performance metrics work
 - Explain how to do nested cross-validation with scikit-learn
 - Explain why one-hot encoding of categorical features is useful/required
 - Explain how to use pipelines in scikit-learn and why they are useful
-

<coding_lab_15 (Day 6 short 1)>

- Implement scikit-learn classifiers yourself
 - Implement nested cross-validation in scikit-learn
 - Implement a pipeline in scikit-learn
-

- Explain how to build a model in Keras
- Explain what the softmax function does
- Know that you can wrap keras models as scikit-learn objects and use them in pipelines
- Explain that in practice you use the weights of the neural network that perform best on the validation set (i.e. you don't train for a set number of epochs)

<coding_lab_17 (Day 6 short 2)>

- Implement simple neural network training in Keras
 - Implement a convolutional neural network in Keras
 - Know how to wrap a keras model for use with scikit-learn hyperparameter estimation
-

Project

- Produce in-depth (pseudo)code to make scikit-learn pipelines
- Implement nested cross-validation for scikit-learn
- Explain how a Random Forest works
- Independently perform an extra step of model tuning